UNIVERSITÀ DELLA CALABRIA
DIPARTIMENTO DI
INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA
E SISTEMISTICA
DIMES

Corso di Laurea Ingegneria Informatica:

Artificial Intelligence & Machine Learning

Progetto Information Retrivial e Natural Language Processing

Comprendere, Valutare e Generare Umorismo con i LLM

Professore:

Andrea Tagarelli

Candidati:

Jacopo Alfonso Le Pera

Mat. 247521

Silvio Emanuele Liparoti

Mat. 242663

ANNO ACCADEMICO 24/25

Indice

Indice **1**

1 Introduzione e obiettivi del progetto **3**

1.1 Problematiche affrontate 3

1.1.1 Motivazioni alla base del problema 5

Possibili soluzioni per il caso d'uso 6

1.2 Architettura complessiva del progetto 7

2 Dati utilizzati e annotazione automatica **10**

2.1 Dataset scelti 10

Processing dei dati 11

2.2 Produzione di dati annotati attraverso tecniche self-supervised . 11

2.2.1 Definizione delle categorie di umorismo 11

2.2.2 Mixture di LLM per la regressione su categorie di umorismo 12

2.2.3 Modelli utilizzati 12

Funzione di classificazione delle battute: prompting e parsing dei risultati in JSON 13

2.2.4 Definizione di una misura di consenso 15

Metrica LPR 15

2.2.5 Valutazione qualitativa degli score ottenuti 18

Confronto con il modello Llama3 18

3 Finetuning di un modello BERT-based per la regressione **20**

3.1 Scelta del modello BERT 20

3.1.1 Problematiche relative all'ambiente di esecuzione 20

3.2 Preprocessing dei dati 21

3.3 Architettura del regressore ALBERT 21
3.4 Addestramento del modello 22
3.4.1 Strategia di training 22
3.4.2 Analisi qualitativa 23
3.5 Salvataggio e utilizzo del modello 27

4 Finetuning della generazione tramite Reinforcement Learning 28
4.1 Architettura del sistema 28
4.2 Definizione della funzione di ricompensa 29
4.2.1 Parametri utilizzati per il Reward Model 30
4.3 Configurazione del fine-tuning: LoRA e PPO 30
4.4 Training e Prompting 32
4.5 Risultati dell'addestramento 34
4.6 Confronto RL vs modello base 35

5 Conclusioni e sviluppi fututri 37

6 Demo del sistema 39
6.1 Modalità di utilizzo 39
6.2 Esempi di interazione 40
6.3 Accesso alla demo 41

Capitolo 1

Introduzione e obiettivi del progetto

Negli ultimi anni, i Large Language Models (LLM) hanno rivoluzionato il panorama dell'elaborazione del linguaggio naturale, mostrando una straordinaria capacità di generare testi coerenti, creativi e dal contesto rilevante. Tuttavia, una delle sfide ancora aperte è la comprensione dell'umorismo, una forma di comunicazione estremamente complessa, fortemente influenzata da fattori culturali, soggettivi e linguistici.
Il presente lavoro si propone di esplorare la capacità dei LLM di generare battute, analizzando la qualità dell'umorismo prodotto e cercando di migliorarne l'efficacia attraverso un processo a più fasi. L'umorismo, infatti, rappresenta un terreno interessante sia dal punto di vista linguistico sia da quello cognitivo: generare una battuta efficace richiede il riconoscimento di schemi, ambiguità, giochi di parole e spesso la gestione di contenuti potenzialmente controversi.

1.1 Problematiche affrontate

Uno degli aspetti più affascinanti, ma al tempo stesso problematici, dei modelli linguistici di grandi dimensioni (LLM) è che a differenza di compiti più strutturati come la classificazione o la traduzione, la generazione umoristica richiede un livello di comprensione semantica, pragmatica e culturale che va oltre la semplice manipolazione sintattica del linguaggio.

Nel nostro progetto abbiamo deciso di concentrare l'attenzione su questo limite specifico: la difficoltà dei LLM nel produrre battute realmente divertenti, originali e coerenti con le richieste dell'utente. Nonostante la loro competenza nel generare frasi grammaticalmente corrette, i modelli spesso falliscono nel cogliere le sfumature tipiche dell'umorismo umano, generando contenuti poco divertenti, privi di senso, e in alcuni casi, rifiutandosi del tutto di rispondere a certi prompt che possono risultare eticamente scorretti.

Per evidenziare concretamente queste difficoltà, abbiamo condotto diversi test empirici interagendo direttamente con modelli linguistici in stile chat. In alcuni casi, i modelli hanno generato battute forzate, grammaticalmente corrette ma prive di impatto umoristico; in altri, hanno evitato di rispondere per via di restrizioni legate alla sicurezza o all'allineamento etico.

Figura 1.1: Esempi di battute generate da un LLM in risposta a richieste di generazione di contenuto umoristico.

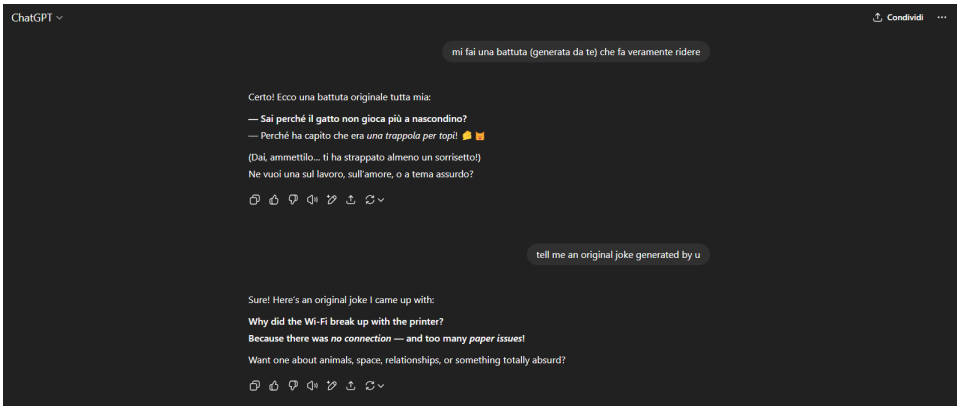

Figura 1.2: Esempi aggiuntivi: generazione umoristica con risultati discutibili.

Come mostrano gli esempi in Figura 1.1 e Figura 1.2, i LLM, se interrogati con prompt generici come *"Genera una battuta originale"* o *"Tell me a joke you came up with"*, producono spesso battute basate su giochi di parole elementari, prevedibili o già sentiti. Ad esempio, si nota il ricorso a colmi classici ("qual è il colmo per un elettricista..."), oggetti parlanti ("il frigorifero che dà consigli di vita") e metafore molto semplici ("una trappola per topi").
Benché formalmente e semanticamente corrette, queste battute risultano generalmente poco incisive dal punto di vista comico e mostrano un pattern ripetitivo nella struttura: una domanda seguita da una risposta prevedibile e priva di reale incongruità semantica o sorpresa.
Questo comportamento evidenzia un problema chiave soprattutto per quanto riguarda la nostra lingua madre, l'italiano, ma anche nella lingua inglese i risultati non sono quelli sperati: i modelli non possiedono una vera comprensione dell'umorismo, ma applicano pattern appresi da grandi corpus testuali, senza una reale valutazione qualitativa. Questo limite ha motivato lo sviluppo del nostro sistema, volto a fornire strumenti per la categorizzazione, valutazione e ottimizzazione della comicità generata.

1.1.1 Motivazioni alla base del problema

Le difficoltà che i modelli linguistici incontrano nella generazione umoristica non sono casuali, ma derivano dalla natura stessa del loro funzionamento. I LLM non possiedono una comprensione semantica profonda, né una coscienza culturale o un'intuizione sociale: essi apprendono correlazioni statistiche tra parole osservando enormi quantità di dati testuali. In altre parole, non "capiscono" realmente l'umorismo, ma ne imitano la forma sulla base di schemi ricorrenti.
Questo implica che, se una battuta o una struttura comica non è sufficientemente rappresentata nei dati di addestramento, è probabile che il modello non riesca a riprodurla o valutarla correttamente. Al contrario, tende a replicare stereotipi o pattern visti molte volte, senza senso critico né sensibilità contestuale. Ne risulta una comicità superficiale, talvolta ripetitiva, spesso prevedibile e poco creativa.

Oltre alle limitazioni dei modelli, va considerata la complessità intrinseca dell'umorismo stesso. L'umorismo è infatti una forma di comunicazione non razionale, altamente soggettiva e fortemente influenzata dal contesto culturale, linguistico e personale. Ciò che risulta divertente per un individuo potrebbe risultare banale, incomprensibile o addirittura offensivo per un altro.
Questa variabilità rende estremamente difficile la formalizzazione e l'automatizzazione dell'umorismo. Mancano metriche oggettive e universalmente accettate per valutarne la qualità, e non esistono benchmark standardizzati che possano guidare l'addestramento o la validazione di modelli generativi in questo ambito. Di conseguenza, risulta complesso sia valutare la comicità di una battuta generata, sia fornire una ricompensa efficace per algoritmi di apprendimento basati su rinforzo.
Questa distanza tra "simulazione" e "comprensione", e tra "output testuale" e "effetto percepito", rappresenta la motivazione principale del nostro lavoro: se i LLM non sanno valutare l'umorismo che generano, dobbiamo fornire loro un sistema di supporto che permetta di misurarlo, interpretarlo e, se possibile, correggerlo. Solo così possiamo costruire un processo iterativo di miglioramento della comicità, attraverso metriche, modelli regressori e tecniche di ottimizzazione mirata.

Possibili soluzioni per il caso d'uso

In un contesto così complesso e soggettivo come quello dell'umorismo, non sempre è possibile definire parametri oggettivi e universali per valutarne la qualità. L'assenza di metriche standard rende difficile costruire un framework rigoroso per l'addestramento e la valutazione dei modelli generativi, specialmente in compiti dove l'obiettivo è produrre contenuti divertenti, creativi o ironici.
Tuttavia, esistono approcci alternativi che possono fornire una stima indiretta o parziale della presenza di elementi umoristici. Uno di questi consiste nell'utilizzare risorse linguistiche come *keyword umoristiche*, cioè insiemi di parole frequentemente associate a battute, giochi di parole, doppi sensi o sarcasmo. Un altro approccio si basa sugli *embeddings contestuali*, sfruttando le rappre-

sentazioni dense dei testi per individuare similarità con battute etichettate o esempi pre-classificati.
Nel nostro lavoro abbiamo scelto di affrontare il problema proponendo una metrica personalizzata che possa fungere da surrogato oggettivo in assenza di annotazioni manuali affidabili. Tale metrica, da noi sviluppata, si basa sul concetto di *consenso tra modelli* e sarà introdotta nei capitoli successivi. Essa prende il nome di **LPR** e costituisce un elemento centrale nel nostro sistema di valutazione automatica, proponendosi come **proxy per la valutazione umana**.

1.2 Architettura complessiva del progetto

Alla luce delle problematiche evidenziate e delle caratteristiche salienti dell'umorismo, abbiamo progettato un'architettura modulare in grado di affrontare il problema su più livelli, con l'obiettivo di migliorare la generazione comica attraverso una pipeline composta da valutazione, modellazione e ottimizzazione.
Il primo componente del sistema è una **mixture di modelli linguistici**, basata su tre LLM open-source (LLaMA, Mistral e Gemma), utilizzata per annotare automaticamente un dataset di battute secondo sei categorie umoristiche da noi definite e che verranno introdotte nella prossima sezione. Ogni modello produce una propria distribuzione di punteggi per ciascuna categoria, permettendo così una classificazione multilabel delle battute.
Successivamente, viene introdotto un meccanismo di **consenso tra modelli**, finalizzato a ridurre la variabilità tra output e ad aumentare la coerenza delle etichette prodotte. Questo meccanismo si concretizza in una metrica proprietaria, da noi sviluppata, denominata **LPR**, che verrà descritta nel dettaglio nei capitoli successivi.
Una volta ottenuto un dataset annotato in modo coerente, il passo successivo è la costruzione di un **modello regressore** capace di prevedere automaticamente i punteggi associati a ciascuna categoria, a partire dal solo testo della battuta. Per questo scopo abbiamo utilizzato il modello **ALBERT**, noto per

essere un modello BERT-like leggero e con buona capacità di generalizzare in contesti a risorse limitate.

Infine, le informazioni ottenute nei passaggi precedenti vengono sfruttate per guidare il miglioramento della generazione di un LLM (nello specifico, Mistral), attraverso una fase di **fine-tuning tramite Reinforcement Learning**. In particolare, adottiamo la tecnica PPO (Proximal Policy Optimization), costruendo una funzione di ricompensa che premia la generazione di battute capaci di ottenere punteggi elevati in almeno una delle categorie umoristiche predefinite.

Questa architettura qui raffigurata ci consente di combinare modelli di generazione, modelli di valutazione e strategie di ottimizzazione in un'unica pipeline coerente, finalizzata al miglioramento della qualità dell'umorismo prodotto in modo automatico.

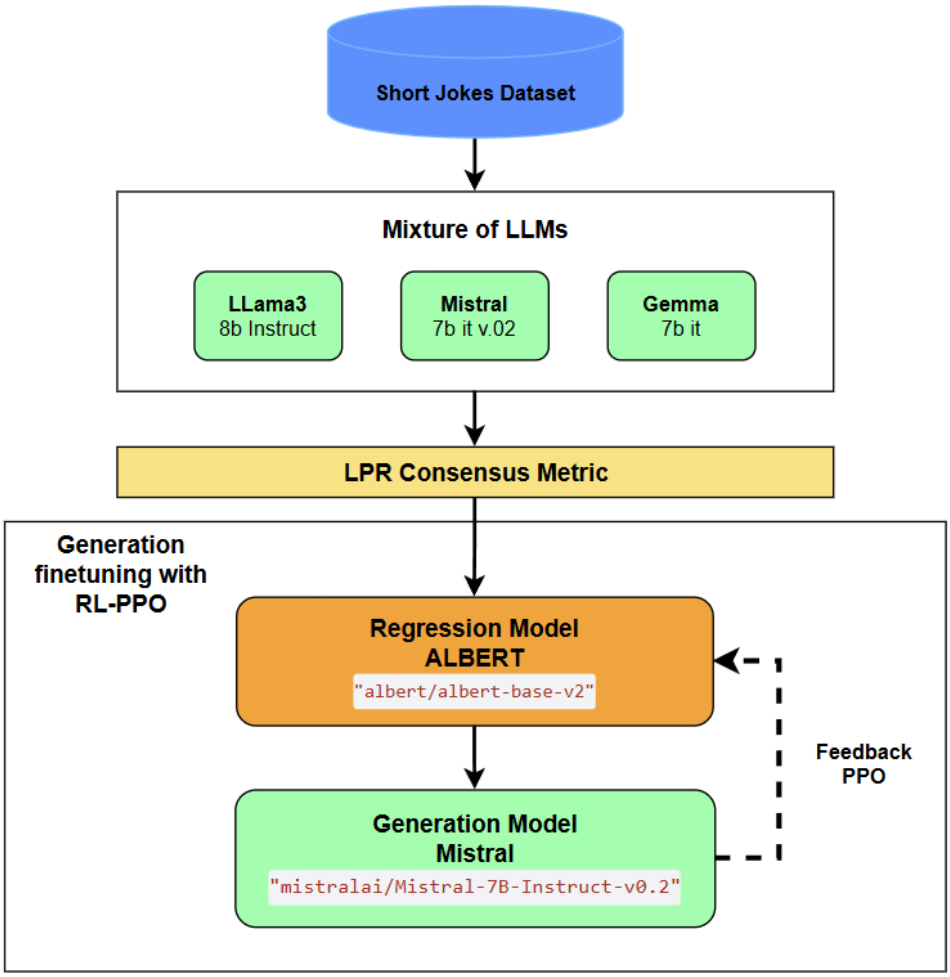

Figura 1.3: Pipeline di progetto

Dunque, la struttura della pipeline segue queste tre fasi operative: si parte dalla categorizzazione automatica, si passa per la modellazione degli score tramite regressione, e si conclude con l'ottimizzazione della generazione tramite reinforcement learning. Le conclusioni finali forniscono una sintesi dei risultati, un'analisi delle criticità incontrate e alcune proposte per possibili sviluppi futuri.

Capitolo 2

Dati utilizzati e annotazione automatica

2.1 Dataset scelti

Per l'addestramento e la valutazione dei nostri modelli abbiamo scelto di utilizzare il dataset *Short Jokes*[1], una raccolta pubblicamente disponibile su Kaggle di oltre 200.000 battute in lingua inglese e di facile lettura poichè composto unicamente dalle colonne ID e joke. Il dataset si presta molto bene allo studio computazionale dell'umorismo, poiché contiene battute di varia natura e complessità, in forma breve e autoconclusiva. Queste caratteristiche permettono una valutazione relativamente isolata del contenuto comico, senza dover gestire contesto narrativo esteso.

Proprio per questo motivo abbiamo scelto di non includere nella trattazione il dataset *Jester*[2]; sebbene lo avessimo inizialmente preso in considerazione, abbiamo scelto di non includerlo nella pipeline principale per motivi legati alla qualità e struttura dei dati. Il dataset *Jester*, riformulato anche su Hugging Face in versione pre-processata[3], presenta numerose colonne interessanti — come , `rating`, `joke_reasoning_steps_llama70b`, `mean_score_user`, `stdev_score_user`, `Z_rating` — che teoricamente lo renderebbero adatto a

[1] https://www.kaggle.com/datasets/abhinavmoudgil95/short-jokes

[2] https://www.kaggle.com/datasets/vikashrajluhaniwal/jester-17m-jokes-ratings-dataset

[3] https://huggingface.co/datasets/SeppeV/rated_jokes_dataset_from_jester

studi su preferenze umoristiche personalizzate. Tuttavia, un'analisi preliminare ha mostrato che il dataset è altamente ridondante: contiene circa 1.76 milioni di righe, ma queste corrispondono a soli 140 testi di barzellette uniche, ognuna valutata più volte da centinaia di utenti. Questa densità di votazioni, pur utile per esperimenti su modelli di preferenza o collaborative filtering, non si adatta bene al nostro obiettivo, ovvero la categorizzazione semantica automatica su vasta scala. Per questo motivo, abbiamo preferito optare per *Short Jokes*, che ci garantisce un alto numero di esempi unici, maggiore varietà di stile e struttura, e una maggiore compatibilità con i modelli di classificazione multilabel e regressione che abbiamo sviluppato. Non escludiamo, però, che il dataset Jester potrebbe essere integrato in futuro per raffinare la fase di training del reward model.

Processing dei dati

Dopo aver caricato il dataset *shortjokes.csv* abbiamo rimosso le battute contenenti valori nulli o non testuali e filtrato il dataset in base alla lunghezza, mantenendo solo le battute comprese tra 25 e 250 caratteri. Questo range consente di escludere da un lato battute troppo corte (che potrebbero essere poco informative) e dall'altro lato battute troppo lunghe (più simili a brevi storie o aneddoti). Dal dataset così ripulito, è stato poi campionato un sottoinsieme di 10.000 battute da sottoporre all'annotazione automatica.

2.2 Produzione di dati annotati attraverso tecniche self-supervised

2.2.1 Definizione delle categorie di umorismo

Abbiamo definito sei categorie principali di umorismo, ciascuna delle quali rappresenta un tipo distinto di comicità:

- **Wordplay**: giochi di parole, doppi sensi.
- **Absurdity**: situazioni paradossali o illogiche.
- **CulturalReference**: riferimenti a contesti pop, storici, o sociali.

- **Relatable**: situazioni quotidiane con cui è facile identificarsi.
- **Offensive**: contenuti provocatori o scorretti.
- **EdgyContent**: battute di tipo dark-humor, borderline, potenzialmente ambigue.

La scelta di queste sei categorie nasce dal desiderio di coprire un ampio spettro dell'umorismo, evitando sovrapposizioni e cercando una suddivisione chiara e mutuamente esclusiva. Questo consente una classificazione multilabel più pulita e una regressione più interpretabile.

2.2.2 Mixture di LLM per la regressione su categorie di umorismo

Abbiamo impiegato tre modelli LLM open-source — LLaMA 3, Mistral e Gemma — per generare in modo automatico punteggi di appartenenza alle sei categorie umoristiche. A ciascun modello è stato fornito un prompt che chiede di valutare la battuta per ogni categoria su una scala da 0 (assente) a 1 (molto presente). L'idea alla base di questa strategia è sfruttare la complementarità tra i modelli per generare una distribuzione più robusta, e ottenere così un sistema di etichettatura automatica più affidabile rispetto all'uso di un singolo modello.

2.2.3 Modelli utilizzati

Come già accennato, per la fase di annotazione automatica abbiamo scelto tre modelli open-source di tipo *instruction-following* pre-addestrati:

- `mistralai/Mistral-7B-Instruct-v0.2`[4]
- `meta-llama/Meta-Llama-3-8B-Instruct`[5]
- `google/gemma-7b-it`[6]

[4] https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2
[5] https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct
[6] https://huggingface.co/google/gemma-7b-it

Tutti i modelli sono stati utilizzati e gestiti tramite pipeline Hugging Face Transformers. La scelta di adottare modelli eterogenei ci ha permesso di confrontare e mediare valutazioni soggettive da parte di architetture diverse, migliorando la robustezza della classificazione multilabel.
I modelli sono stati chiamati in modo iterativo su ciascuna battuta, ricevendo in input un prompt strutturato per restituire uno score numerico da 0 a 10 per ciascuna categoria umoristica.

Funzione di classificazione delle battute: prompting e parsing dei risultati in JSON

Di seguito viene riportata la funzione di classificazione, punto cardine del nostro primo notebook Kaggle:

```python
# Classify jokes with scores in structured JSON
def classify_jokes_with_scores(df, pipe, model_key, batch_size=32):
    prompts = []
    for joke in df["jokeText"]:
        category_list = "\n".join([f'- "{cat}": {desc}' for cat, desc in categories.items()])
        prompt = f"""You are a humor expert.
        Rate how much the following joke fits each of these humor categories (scale: 0 to 10).

        Categories:
        - Edgy Content: Sex, taboos, death, shock
        - Cultural Reference: Politics, celebrities, pop culture
        - Wordplay: Puns, linguistic ambiguity
        - Absurdity: Illogical logic, surrealism, nonsense
        - Relatable: Everyday life, technology, relationships, self-irony
        - Offensive Humor: Insults, stereotypes, sarcasm, humiliation

        Respond ONLY with a JSON object.
        You must not include any other text, just assign the scores using the json format.

        Here's an example:
        {{"Edgy Content": 6, "Cultural Reference": 8, "Wordplay": 1, "Absurdity": 0, "Relatable": 3, "Offensive Humor": 9}}

        Rate this joke:
        Joke:
        \"\"\"{joke}\"\"\"
        """

        prompts.append(prompt)
```

```python
    # Initialize score columns
    for cat in categories:
        df[f"{model_key}_{cat}"] = None

    for i in range(0, len(prompts), batch_size):
        batch = prompts[i:i + batch_size]
        print(f"🌀 [{model_key}] Batch {i // batch_size + 1}")

        outputs = pipe(
            batch,
            max_new_tokens=100,
            do_sample=False,
            batch_size=batch_size,
            temperature=0.0
        )

        for j, output in enumerate(outputs):
            text = output[0]["generated_text"]
            #print(text)
            try:
                scores = extract_last_json_from_text(text)
                for cat in categories:
                    df.at[i + j, f"{model_key}_{cat}"] = scores.get(cat, 0)
            except Exception as e:
                print(f"⚠️ JSON parsing error in batch {i // batch_size + 1}, item {j}: {e}")
                print(f"📝 Model output:\n{text}\n{'-'*60}")
                for cat in categories:
                    df.at[i + j, f"{model_key}_{cat}"] = None

    return df

# modelli
models_to_use = {
    "mistral": "mistralai/Mistral-7B-Instruct-v0.2",
    #"llama3": "meta-llama/Meta-Llama-3-8B-Instruct",
    #"gemma": "google/gemma-7b-it",
}
```

Figura 2.1: Funzione di classificazione con scoring JSON restituito dal modello

La funzione riportata nella Figura 2.1 rappresenta il nucleo operativo della pipeline di annotazione. Essa prende in input un dataframe di battute, una pipeline di generazione basata su un modello LLM, e un identificatore del modello.

Per ogni battuta viene costruito dinamicamente un prompt dettagliato che descrive le sei categorie umoristiche da valutare. Il modello è quindi chiamato a restituire, in formato JSON, uno score da 0 a 10 per ciascuna categoria. Il

risultato viene poi inserito nel dataframe, creando colonne separate per ogni categoria e per ogni modello.
L'approccio presenta diversi vantaggi: è scalabile, interpretabile e facilmente estensibile ad altri modelli o categorie. Tuttavia, essendo basato sull'output libero del modello, richiede un parsing robusto e include meccanismi di gestione degli errori per garantire affidabilità nel processo di raccolta dati.

2.2.4 Definizione di una misura di consenso

Abbiamo introdotto la metrica **LPR** (Le Pera - Liparoti), che rappresenta una misura sintetica del consenso tra modelli rispetto alla presenza di almeno una categoria umoristica significativa.
Dato uno score multilabel per ogni modello, la LPR valuta il numero di categorie con punteggio superiore a una soglia (es. 0.6) e la coerenza tra i modelli nel segnalarle. Questa metrica funge da surrogato della qualità percepita, ed è impiegata successivamente come base per l'addestramento del reward model.

Metrica LPR

La metrica **LPR** (Le Pera - Liparoti) da noi introdotta restituisce un unico punteggio consensuale per ogni categoria di umorismo, calcolato a partire dalle valutazioni assegnate da tre modelli LLM (LLaMA3, Mistral, Gemma). L'obiettivo è ottenere una stima robusta anche in presenza di disaccordi tra i modelli, premiando i casi di coerenza e penalizzando le divergenze estreme.
Il concetto chiave alla base della metrica è quello di *disaccordo controllato*. Supponiamo che due modelli assegnino punteggi simili per una certa categoria (ad esempio 7 e 6), mentre il terzo assegna un valore molto distante (ad esempio 1). In questo caso, si rileva una **deviazione massima** significativa tra i punteggi. La metrica valuta tutte le coppie di modelli e, se almeno due di essi sono in *accordo accettabile* — ovvero la loro differenza rientra entro una soglia predefinita chiamata `max deviation` (tipicamente fissata a 2) — allora si procede con una media pesata limitata solo ai modelli coerenti tra loro.
Più nel dettaglio, le casistiche si articolano nei seguenti punti:

1. **Accordo: Media ponderata.** In caso di totale accordo tra i modelli lo score risultante è rappresentato dalla media ponderata, basata su valutazioni empiriche di qualità (benchmark): LLaMA3 (0.5), Mistral (0.3), Gemma (0.2).

2. **Disaccordo parziale: consenso sui modelli in accordo.** Per ogni categoria: se almeno due modelli hanno una differenza $\leq$ `max deviation`, si considera che esista un accordo e si calcola la media pesata su quei modelli.

3. **Disaccordo forte**: se nessuna coppia di modelli rispetta la soglia di coerenza, si assume che il consenso manchi. In questo caso, si ricorre al punteggio fornito dal **modello con il peso più alto** (es. LLaMA3), considerato più affidabile.

La soglia `max deviation` consente di regolare la tolleranza al disaccordo: valori alti rendono la metrica più permissiva e portano più frequentemente al calcolo della media tra tutti i modelli; valori bassi, invece, privilegiano l'affidabilità del singolo modello dominante. Infine, è possibile personalizzare completamente il comportamento della metrica, ad esempio assegnando pesi uniformi o includendo una componente di casualità in caso di disaccordo totale.
Di seguito, riportiamo l'implementazione completa della funzione `compute_lpr`.

```python
def compute_lpr(merged_df, category, weights=None, max_deviation=3):
    if weights is None:
        weights = {"llama3": 0.5, "mistral": 0.3, "gemma": 0.2}

    available_models = [model for model in weights if f"{model}_{category}" in merged_df.columns]
    usable_weights = {model: weights[model] for model in available_models}

    method_counts = {
        "media_ponderata_completa": 0,
        "media_ponderata_filtrata": 0,
        "fallback_llama3": 0
    }

    scores = []

    for _, row in merged_df.iterrows():
        ratings = {m: row[f"{m}_{category}"] for m in available_models}
        values = list(ratings.values())
        models = list(ratings.keys())

        # Calcola disaccordi tra le coppie
        pairs = [(m1, m2) for i, m1 in enumerate(models) for m2 in models[i+1:]]
        disagreements = {(m1, m2): abs(ratings[m1] - ratings[m2]) for m1, m2 in pairs}
        strong_disagreement = [pair for pair, dist in disagreements.items() if dist > max_deviation]

        if len(strong_disagreement) == 0:
            # Caso 1: consenso accettabile → media pesata
            method_counts["media_ponderata_completa"] += 1
            score = sum(ratings[m] * usable_weights[m] for m in models)
        else:
            # Caso 2: disaccordo → filtro sui modelli vicini alla media
            mean_value = np.mean(values)
            filtered_models = [m for m in models if abs(ratings[m] - mean_value) <= max_deviation + 1]
            if len(filtered_models) >= 2:
                method_counts["media_ponderata_filtrata"] += 1
                total_weight = sum(usable_weights[m] for m in filtered_models)
                score = sum(ratings[m] * usable_weights[m] for m in filtered_models) / total_weight
            else:
                # Caso 3: fallback → modello col peso maggiore
                method_counts["fallback_llama3"] += 1
                model_with_max_weight = max(usable_weights.items(), key=lambda x: x[1])[0]
                score = ratings[model_with_max_weight]

        scores.append(round(score))

    return pd.Series(scores, index=merged_df.index), method_counts
```

Figura 2.2: Funzione di calcolo dello score LPR per una singola categoria

La funzione riportata nella Figura 2.2 implementa la metrica LPR, e si applica riga per riga al dataframe contenente le valutazioni prodotte dai tre modelli. Per ogni categoria, essa considera le valutazioni e calcola una media ponderata in base ai pesi assegnati, tenendo conto della tolleranza definita dal parametro `max_deviation`.

In presenza di disaccordo limitato, il punteggio finale riflette un compromesso tra i modelli. In caso contrario, viene applicato un filtro per escludere outlier, oppure si privilegia il modello considerato più affidabile (LLaMA3), garantendo così un output robusto anche in contesti rumorosi.

Questo approccio ha permesso di ottenere score più stabili e credibili rispetto a una semplice media aritmetica, e ha rappresentato la base per l'etichettatura

finale usata nelle fasi successive del progetto.

2.2.5 Valutazione qualitativa degli score ottenuti

L'analisi qualitativa delle predizioni mostra che le valutazioni ottenute tramite la mixture di LLM riflettono con buona coerenza il contenuto della battuta. In particolare, battute provocatorie o con riferimenti culturali espliciti ottengono punteggi alti nelle categorie Offensive e CulturalReference, mentre giochi di parole semplici tendono a emergere chiaramente nella categoria Wordplay.
Questo conferma che, pur in assenza di supervisione umana, i modelli riescono a catturare dimensioni differenti del comico attraverso un processo di autoannotazione.

Confronto con il modello Llama3

Come caso d'esempio significativo per approfondire l'efficacia della metrica proposta, abbiamo analizzato una battuta su Adolf Hitler presente nel dataset, contenente un gioco di parole che mescola riferimenti storici e black humour. Il testo della battuta è il seguente:
What was Hitler's favourite aquatic animal? Adolfin.
Di seguito vengono riportati gli score LPR per le categorie e quelli assegnati dai tre modelli in formato tabellare

Modello	Edgy Content	Cultural Reference	Wordplay	Absurdity	Relatable	Offensive Humor
LPR	9	8	1	7	0	9
LLaMA	0	8	9	0	0	0
Mistral	10	0	1	8	0	9
Gemma	8	9	0	6	2	8

Tabella 2.1: Score per categoria della battuta analizzata

Questo confronto mette in evidenza il valore della metrica LPR basata sul consenso: rispetto ai modelli Gemma e Mistral, LPR riconosce in modo più equilibrato sia il riferimento storico, sia il gioco di parole tra "dolphin" e "Adolf", riflettendoli in punteggi coerenti; anche in confronto al modello LLaMA3, che nei benchmark risulta essere il più performante tra quelli scelti, LPR dimostra una maggiore sensibilità nel cogliere il contenuto offensivo e gli elementi di dark humour presenti nella battuta, includendo inoltre la componente surreale

tipica della categoria “Absurdity”. Questo suggerisce che la metrica LPR riesce a catturare sfumature del contenuto comico che i modelli singoli tendono a trascurare.

Capitolo 3

Finetuning di un modello BERT-based per la regressione

3.1 Scelta del modello BERT

3.1.1 Problematiche relative all'ambiente di esecuzione

Nel contesto della nostra pipeline, uno degli obiettivi principali è la possibilità di ottenere una previsione automatica dei punteggi umoristici (annotati precedentemente con i LLM) a partire dal solo testo della battuta. Per questo motivo si è deciso di ricorrere a un modello Transformer pre-addestrato, fine-tunato come regressore multilabel.

Tra le diverse scelte disponibili, i vincoli pratici legati all'ambiente di esecuzione (in particolare l'utilizzo della piattaforma Kaggle, dotata di GPU con memoria limitata) hanno reso necessario l'uso di un modello leggero. La scelta è ricaduta su **ALBERT-base-v2**[1] , una variante di BERT che presenta pesi condivisi tra i layer e una riduzione della dimensione degli embedding.

ALBERT è in grado di mantenere buone performance su compiti di NLP mantenendo al tempo stesso un ingombro computazionale ridotto. Si è quindi rivelato adatto a operare in ambienti low-resource, senza sacrificare significativamente la capacità di rappresentare il contenuto semantico delle battute.

[1] `https://huggingface.co/albert-base-v2`

3.2 Preprocessing dei dati

Per il training del regressore, è stato utilizzato il dataset da noi estratto attraverso le procedure precedentemente descritte `aggregated_scores_LPR_10000.csv`, contenente 10.000 battute annotate automaticamente da tre modelli LLM (LLaMA3, Mistral, Gemma) mediante la metrica di consenso LPR.
I punteggi presenti nelle colonne relative alle sei categorie umoristiche sono stati:

- ripuliti (rimozione di spazi nei nomi delle colonne),
- filtrati (rimozione dei record privi di testo o di punteggi validi).

Inoltre:

- Le battute sono state tokenizzate utilizzando `AutoTokenizer`[2] compatibile con ALBERT.
- È stato applicato padding automatico e generata la `attention mask`, in linea con l'input richiesto dai modelli Transformer.
- Gli score di ciascuna categoria sono stati normalizzati nel range [0, 1].

Questa normalizzazione si è resa necessaria per adattarsi all'output della funzione `Sigmoid`, che produce valori compresi tra 0 e 1. Il dataset è stato poi suddiviso in `train set` e `test set`, mantenendo una proporzione 80/20 e garantendo la presenza bilanciata di tutte le categorie.

3.3 Architettura del regressore ALBERT

Il modello di regressione sviluppato si basa sull'utilizzo di ALBERT come encoder testuale. Per ciascuna battuta, il modello produce un embedding del token speciale [CLS], che viene poi elaborato da una semplice rete neurale fully-connected.
L'architettura può essere così descritta:

[2] `https://huggingface.co/docs/transformers/model_doc/auto`

- **Encoder** → ALBERT: produce una rappresentazione vettoriale della battuta (dimensione ∼768).
- **Regressore neurale**:
  - `Linear` (768 → 128),
  - `ReLU`,
  - `Linear` (128 → 6),
  - `Sigmoid` → output $\in [0, 1]$.

Ogni uscita corrisponde a una delle sei categorie umoristiche. I valori ottenuti sono poi moltiplicati per 10 per riportare il punteggio nel dominio originale.
L'approccio scelto, pur essendo semplice, ha il vantaggio di sfruttare la potenza espressiva dei Transformer per l'estrazione delle feature semantiche, delegando al regressore solo il mapping finale.

3.4 Addestramento del modello

3.4.1 Strategia di training

L'addestramento è stato svolto per 5 epoche, con batch size pari a 16 e learning rate fissato a $2 \cdot 10^{-5}$. Il modello è stato ottimizzato tramite l'algoritmo `AdamW`[3] , una variante del classico ottimizzatore Adam che introduce una regolarizzazione L2 sui pesi (weight decay).
Come funzione obiettivo è stata impiegata la `MSELoss`[4] tra i punteggi reali e quelli predetti, in quanto adatta a problemi di regressione continua. L'addestramento è stato implementato manualmente in PyTorch, iterando sui batch del dataset e aggiornando i pesi in modo esplicito.
Il training ha mostrato una convergenza stabile e regolare già a partire dalle prime epoche.
Per la valutazione delle performance è stato utilizzato il test set. Per ciascuna delle sei categorie, sono state calcolate due metriche fondamentali:

[3] `https://huggingface.co/docs/transformers/main_classes/optimizer_schedules#transformers.AdamW`

[4] `https://pytorch.org/docs/stable/generated/torch.nn.MSELoss.html`

- **Mean Squared Error (MSE)** – errore quadratico medio
- **Mean Absolute Error (MAE)** – errore assoluto medio

I risultati ottenuti sono riassunti di seguito (valori esemplificativi):

```
Wordplay → MSE: 0.0124, MAE: 0.082
Relatable → MSE: 0.0157, MAE: 0.095
...
```

3.4.2 Analisi qualitativa

Oltre alle metriche aggregate, è stato condotto un confronto qualitativo tra i valori predetti dal modello e le etichette ottenute tramite metrica LPR. In Figura 3.1, vengono mostrati i grafici *Predicted vs Ground Truth* per le sei categorie umoristiche:

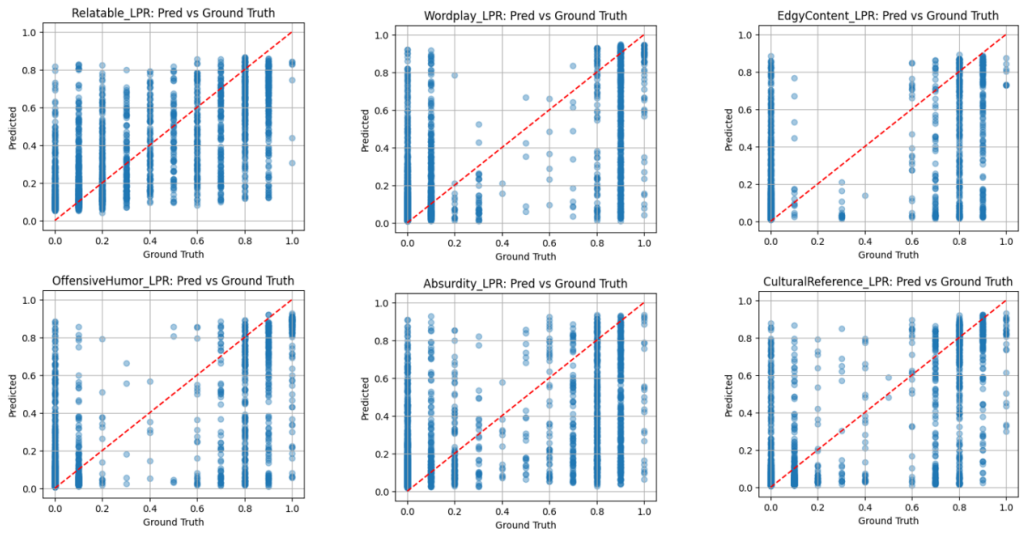

Figura 3.1: Scatter plot Prediction vs Ground Truth per le categorie

Dall'osservazione dei grafici, emerge una prestazione variegata del modello. Alcune categorie, come *Relatable* e *Offensive Humor*, mostrano una discreta aderenza alla diagonale ideale (linea rossa), indicando una certa capacità di apprendere le strutture umoristiche più esplicite o frequenti. Tuttavia, categorie più sottili o linguisticamente complesse, come *Wordplay* e *Cultural Reference*, evidenziano una dispersione significativa, con numerose predizioni distanti dal valore corretto.

Questo comportamento può essere attribuito a vari fattori. Innanzitutto, l'umorismo è notoriamente difficile da modellare in quanto soggettivo, implicito e culturalmente dipendente. In secondo luogo, il compito affidato al modello è particolarmente ambizioso: prevedere score continui, derivati da un'aggregazione automatica fra tre modelli LLM, usando una rete relativamente compatta come ALBERT. Questa combinazione introduce inevitabilmente rumore nel dataset e aumenta la complessità del problema di regressione.
Nonostante queste difficoltà, il modello ha mostrato coerenza in alcune categorie e buone tendenze di generalizzazione, specialmente nei casi in cui la relazione tra testo e categoria era più evidente. Questo suggerisce che l'approccio è valido come punto di partenza, ma può essere migliorato con modelli più espressivi, supervisioni più pulite o metriche di consenso più sofisticate.
Per valutare il comportamento del regressore in situazioni complesse, abbiamo analizzato qualitativamente alcune predizioni su battute selezionate. Oltre alle metriche aggregate, questa valutazione permette di osservare il grado di coerenza e sensibilità semantica del modello su esempi reali. Si riportano vari esempi testuali particolarmente rappresentativi:

Joke 1

> *"If a stork brings white babies and a crow brings black babies, what brings no babies? A little Swallow."*

```
EdgyContent_LPR: GT = 0.00 | Pred = 0.42
CulturalReference_LPR: GT = 0.10 | Pred = 0.07
Wordplay_LPR: GT = 0.90 | Pred = 0.07
Absurdity_LPR: GT = 0.80 | Pred = 0.33
Relatable_LPR: GT = 0.00 | Pred = 0.13
OffensiveHumor_LPR: GT = 0.00 | Pred = 0.60
```

In questo caso, il modello fallisce nel cogliere la vera natura comica della battuta. La presenza di un doppio senso ("swallow" come uccello ma anche come atto sessuale) avrebbe dovuto condurre a un punteggio alto in *Wordplay* e *Absurdity*, ma entrambi sono fortemente sottostimati. Viene comunque attribuito un punteggio elevato alla categoria *OffensiveHumor*, catturando quindi

in qualche misura il contenuto esplicito della battuta, nonostante non fosse stato appurato dalla metrica di riferimento. L'output dimostra dunque una difficoltà del modello a gestire battute con significati impliciti o doppi.

Joke 2

> *"People are always mistaking things I say as racist. The other day, this guy thought I called him a 'sand nigger.' But what I said was, 'get out of the sand, nigger. Volleyball is a white man's sport.' "*

```
EdgyContent_LPR: GT = 0.90 | Pred = 0.84
CulturalReference_LPR: GT = 0.50 | Pred = 0.48
Wordplay_LPR: GT = 0.00 | Pred = 0.09
Absurdity_LPR: GT = 0.00 | Pred = 0.08
Relatable_LPR: GT = 0.00 | Pred = 0.08
OffensiveHumor_LPR: GT = 1.00 | Pred = 0.91
```

Questa battuta estremamente provocatoria contiene un linguaggio razzista ed esplicitamente offensivo. In questo caso, il modello ha risposto correttamente, riconoscendo correttamente le componenti *Edgy Content* e *Offensive Humor*, dimostrando una certa sensibilità nei confronti di contenuti eticamente problematici. Il punteggio vicino a zero nelle categorie meno pertinenti conferma un comportamento prudente e coerente. Questo esempio mette in luce uno dei punti di forza del modello: la capacità di riconoscere elementi forti e ben marcati, specie quando le categorie semantiche sono chiare e inequivocabili.

Conclusioni qualitative L'analisi conferma che il modello tende a comportarsi meglio su battute "esplicite" o chiaramente categorizzabili (es. contenuti offensivi), mentre fatica su forme di umorismo più sottili, implicite o linguisticamente ambigue. Le difficoltà principali emergono nel riconoscimento del *Wordplay*, dove l'informazione semantica viene trasmessa in modo indiretto o attraverso giochi di significato.

Joke 7

> *"My stalker twisted his ankle, so now I have to walk slower for 2-3 weeks."*

```
EdgyContent_LPR: GT = 0.00 | Pred = 0.74
CulturalReference_LPR: GT = 0.00 | Pred = 0.10
Wordplay_LPR: GT = 0.00 | Pred = 0.11
Absurdity_LPR: GT = 0.00 | Pred = 0.59
Relatable_LPR: GT = 0.90 | Pred = 0.18
OffensiveHumor_LPR: GT = 0.00 | Pred = 0.53
```

Il modello ha assegnato punteggi elevati ad aspetti come l'*Offensive Humor* o l'*Absurdity*, pur in assenza di annotazioni in tal senso. Questo suggerisce una difficoltà nella comprensione semantica del contenuto: parole come *stalker* potrebbero aver innescato un'associazione erronea a contenuti "edgy" o offensivi, sebbene il tono sia ironico ma innocuo.

Joke 8

> *"I didn't go through four years of grad school for this! (I didn't go to grad school.)"*

```
EdgyContent_LPR: GT = 0.00 | Pred = 0.02
CulturalReference_LPR: GT = 0.00 | Pred = 0.34
Wordplay_LPR: GT = 0.80 | Pred = 0.03
Absurdity_LPR: GT = 0.90 | Pred = 0.24
Relatable_LPR: GT = 0.90 | Pred = 0.76
OffensiveHumor_LPR: GT = 0.00 | Pred = 0.01
```

In questo esempio il modello ha mostrato una buona sensibilità sul contenuto *Relatable*, mentre ha completamente mancato la componente di *Wordplay*, che era ben presente nella battuta originale (l'ironia del "finto" grad school). Questo evidenzia la difficoltà del modello nel riconoscere sfumature comiche basate su ambiguità e contraddizione semantica.

Conclusione In generale, il regressore ALBERT ha dimostrato buone capacità di generalizzazione nei casi più lineari, ma fatica a cogliere le sfumature comiche più sottili, specialmente quelle basate su giochi di parole, doppi sensi o costruzioni ironiche implicite.

3.5 Salvataggio e utilizzo del modello

Al termine del training, il modello è stato salvato in formato `state_dict` per poter essere ricaricato successivamente e utilizzato nelle fasi di reinforcement learning.

Inoltre, è stata definita una funzione `predict_scores` che, dato un elenco di battute, produce per ciascuna un vettore di punteggi predetti, normalizzati su scala [0, 10]. Questo modulo è stato pensato come componente integrabile in una futura **reward function** basata su umorismo, usata per guidare la generazione.

```
Modello salvato su /kaggle/working/joke_regressor_state_dict.pt
```

Capitolo 4

Finetuning della generazione tramite Reinforcement Learning

In questo capitolo descriviamo l'ultima fase della pipeline: il miglioramento della generazione comica da parte del modello Mistral attraverso tecniche di Reinforcement Learning, in particolare l'algoritmo PPO [1] (Proximal Policy Optimization), combinato con l'adattamento efficiente dei pesi tramite LoRA[2] (Low-Rank Adaptation). L'obiettivo è spingere il modello generativo a produrre battute che ottengano punteggi alti in almeno una delle categorie umoristiche definite nei capitoli precedenti.

4.1 Architettura del sistema

Il sistema è composto da tre componenti principali:

- **Modello generativo**: Mistral-7B-Instruct, finetunato con LoRA, che produce battute a partire da un prompt utente.
- **Modello valutatore**: ALBERT, già addestrato in precedenza per assegnare punteggi da 0 a 10 alle battute su ciascuna categoria.
- **Modello di riferimento**: una copia congelata del modello base Mistral, utilizzata per calcolare la divergenza KL rispetto al nuovo modello addestrato.

[1] https://arxiv.org/abs/1707.06347
[2] https://arxiv.org/abs/2106.09685

Questo approccio permette di ottimizzare la generazione, favorendo battute più divertenti (secondo la valutazione del modello ALBERT) e mantenendo al contempo la coerenza con lo stile originario.

4.2 Definizione della funzione di ricompensa

Il reward è calcolato combinando tre aspetti:

1. Un reward **base** se almeno una categoria supera una soglia prestabilita (base reward = 1.0). In caso di non raggiungimento della soglia per alcuna categoria, viene assegnato 0.

2. Un bonus legato all'**intensità** della categoria con punteggio massimo

$$\text{bonus_score} = \frac{\max(\text{scores}) - \text{threshold}}{5}$$

   Qui il bonus viene calcolato come score massimo ottenuto dalla risposta per una categoria qualsiasi, a cui viene sottratta la soglia minima, e viene diviso lo scarto per una costante (che fa da fattore di scala).

3. Un bonus legato alla **varietà**, ossia al numero di categorie che superano la soglia

$$\text{bonus_variety} = \alpha \times (n_{\text{cat}} - 1)$$

   Per ogni categoria aggiuntiva che supera la soglia viene assegnato un punteggio prestabilito.

Nel caso in cui il testo generato sia vuoto o non superi nessuna soglia, il reward viene azzerato o penalizzato. In Figura 4.1 è mostrato il calcolo del reward:

```python
def compute_reward(scores, threshold=6.0, alpha=0.25, empty_penalty=-0.5, text=None):
    """
    Calcola il reward per una singola risposta sulla base degli score di categoria.

    - Se nessuna categoria supera la soglia → reward = 0.0
    - Altrimenti:
        - base reward = 1.0
        - bonus_score = (max(scores) - threshold) / 5
        - bonus_variety = α × (n_cat - 1)
    """
    if text is not None and len(text.strip()) == 0:
        return empty_penalty
    scores = np.array(scores)
    above_threshold = scores >= threshold
    n_cat = above_threshold.sum()

    if n_cat == 0:
        return 0.0

    base = 1.0
    bonus_score = max(scores) - threshold
    bonus_score = max(bonus_score, 0.0) / 5.0  #10.0 # cambiato a 5.0 per dare più peso

    bonus_variety = alpha * (n_cat - 1)  # solo extra categorie

    total_reward = base + bonus_score + bonus_variety
    return float(total_reward)
```

Figura 4.1: Funzione di reward utilizzata.

4.2.1 Parametri utilizzati per il Reward Model

I parametri di default del metodo, che abbiamo usato nella nostra pipeline, prevedono una soglia minima pari a 6, un fattore bonus di 0.25 per ogni categoria aggiuntiva che supera la soglia, e un fattore di penalità pari a -0.5 per ogni risposta vuota del modello.
Inoltre, il fattore costante che divide il bonus score è stato cambiato da 10 (che prevedeva l'assegnazione di 1 punto per ogni voto sopra la soglia) a 5, per dare più peso a battute notevolmente oltre soglia.

4.3 Configurazione del fine-tuning: LoRA e PPO

Per ridurre il costo computazionale, è stata utilizzata la tecnica LoRA (Low-Rank Adaptation), che consente di aggiornare solo alcune matrici del modello

tramite una decomposizione a rango ridotto. La configurazione adottata è la seguente:

- `r=8`: indica il rango della decomposizione a basso rango; valori più bassi riducono il numero di parametri addestrabili e quindi il costo computazionale, ma possono limitare la capacità di adattamento.
- `lora_alpha=16`: rappresenta un fattore di scalatura che controlla l'ampiezza dell'aggiornamento appreso. Valori più alti aumentano l'impatto degli aggiornamenti LoRA sul modello finale.
- `lora_dropout=0.1`: introduce un dropout del 10% sugli aggiornamenti LoRA durante l'addestramento, utile per ridurre l'overfitting e migliorare la generalizzazione.
- `target_modules=["q_proj", "v_proj"]`: specifica i moduli dell'architettura Transformer sui quali applicare LoRA. In questo caso sono state scelte le proiezioni di query e value dell'attenzione, poiché incidono fortemente sulla rappresentazione del contesto senza dover aggiornare l'intero modello.

La configurazione scelta rappresenta un approccio moderato rispetto al task affrontato: ci ha permesso di aggiornare solo una piccola parte dei parametri del modello Mistral, riducendo notevolmente la memoria necessaria e accelerando l'addestramento. Tuttavia, gli aggiornamenti possono risultare significativi, se combinati al prompting e al reward model utilizzato. Attraverso diversi test è stato possibile verificare la variazione del comportamento del modello di generazione in base ai parametri scelti.
L'addestramento è stato condotto usando PPO (Proximal Policy Optimization), una tecnica di reinforcement learning. In questo approccio, la loss finale combina tre componenti principali:

- il reward ottenuto dal modello di valutazione, che guida l'ottimizzazione verso comportamenti desiderati;
- il logaritmo della probabilità della sequenza generata, che incentiva la coerenza tra la politica appresa e la distribuzione delle risposte;

- la divergenza KL tra il modello corrente e il modello congelato, utilizzata come termine di regolarizzazione per evitare deviazioni eccessive dal modello di partenza.

4.4 Training e Prompting

Durante l'addestramento, al modello sono stati forniti i seguenti prompt:

- Tell me a Monday morning joke
- Tell me an office joke
- Tell me a one-liner about cats and their laziness
- Make up a joke about work calls
- Make a funny joke about dieting
- Make me laugh with a dad joke about barbecue
- Tell me a short joke about artificial intelligence
- Do you have a joke about Italian stereotypes?
- Make a joke about college students
- Make up a nonsensical joke about Donald Trump
- Make black humor about the September 11, 2001, disaster
- Explain me the history of the Colosseum
- Explain your last joke
- What do you mean?
- How are you

Il prompt utente, invece, viene incapsulato all'interno di un contesto istruttivo tramite la seguente funzione mostrata in Figura 4.2

```python
def make_prompt(user_prompt):
    return f"""<s>[SYSTEM]
        You are a professional comedy writer who specializes in one-liners and dad jokes.
        Your style is irreverent, witty, and playful - sometimes exaggerated, but always clever
and tailored to the user's prompt.
        You never explain the joke. You never break character. Just deliver a short, sharp punch
line that fits the context.
        Always be original, avoid clichés, and keep it brief - one or two lines at most.
        If the user simply asks for a joke, invent one on the spot. Your job is to make people l
augh.
        If the user asks to explain a joke or asks something that is not relevant in the context
of jokes,
        make fun of them, but don't be offensive.
        \n[USER] {user_prompt} . \n [ASSISTANT]</s>"""
```

Figura 4.2: Costruzione del prompt in stile chat.

Nel processo di addestramento, il modello generativo riceve dei prompt predefiniti a cui rispondere in stile comico. Per strutturare correttamente il contesto e istruire il modello a comportarsi come un *comedy writer*, viene definito un template che simula un'interazione in stile chat tra utente e assistente. Questa struttura è conforme a quanto previsto nei modelli `Instruct`-based, ed è composta da tre sezioni:

- [**SYSTEM**]: fornisce istruzioni iniziali al modello su come comportarsi (ad esempio, uno scrittore comico professionista).
- [**USER**]: rappresenta il prompt effettivo fornito dall'utente (es. *"Tell me a Monday morning joke"*).
- [**ASSISTANT**]: è lo spazio in cui il modello genererà la propria risposta[3].

Questi tag sono racchiusi all'interno del token iniziale `<s>` e vengono concatenati in un'unica stringa per guidare il comportamento del modello durante la generazione.
Ad ogni iterazione del training, uno di questi prompt viene selezionato casualmente e trasformato in input per la generazione del modello. La risposta viene poi valutata tramite il reward personalizzato precedentemente delineato, e il ciclo di aggiornamento procede come descritto nella sezione successiva.

[3] `[SYSTEM]`, `[USER]`, `[ASSISTANT]` descritti in `https://huggingface.co/docs/transformers/chat_templating`

Un esempio di generazione dopo l'addestramento è:

> **Prompt**: "Make a joke about work calls"
>
> **Risposta RL**: "If one more work call comes in, I'm charging rent."
>
> **Risposta base**: "My boss called during lunch, so I muted him like my salad."

La risposta dopo aver effettuato il Reinforcement Learning risulta più originale e coerente con lo stile comico desiderato.

4.5 Risultati dell'addestramento

Durante le 1000 iterazioni di PPO, sono stati salvati reward, KL divergence e loss. Il reward medio ha mostrato una crescita iniziale, poi una stabilizzazione. La Figura 4.3 mostra l'andamento del reward nel tempo.

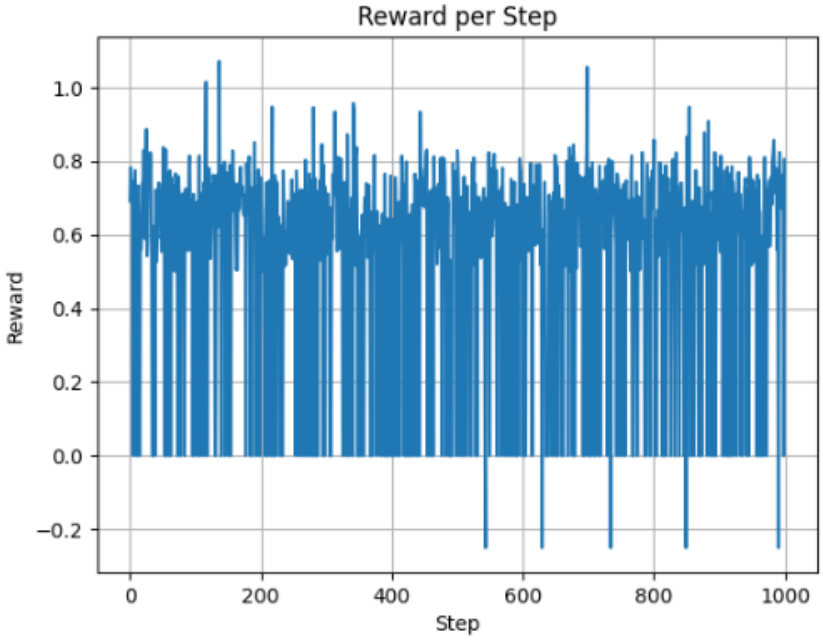

Figura 4.3: Andamento del reward durante il training.

Nel grafico riportato sopra viene mostrata l'evoluzione del *reward* ad ogni step di aggiornamento del modello durante il fine-tuning con Reinforcement Learning. L'asse delle ascisse rappresenta il numero di iterazioni (step), mentre

l'asse delle ordinate rappresenta il valore del reward scalato, secondo quanto definito nella funzione `compute_reward`.
La curva è tendenzialmente irregolare, con oscillazioni frequenti tra valori alti e bassi: questo comportamento è tipico in contesti con generazione stocastica e valutazioni soggettive come quello dell'umorismo. Tuttavia, si può osservare una leggera tendenza all'aumento nella parte centrale e finale, a suggerire che il modello inizia a produrre risposte che riescono a superare la soglia di qualità stabilita con maggiore frequenza.
In particolare, l'aumento della media dei reward nel tempo indica che il modello sta apprendendo a generare battute che soddisfano uno o più criteri di comicità secondo la metrica LPR. Le fluttuazioni sono legate alla casualità dei prompt scelti e alla variabilità insita nella generazione testuale.

In sintesi, il grafico non mostra un miglioramento netto e costante, ma fornisce evidenza che il modello si stia adattando parzialmente al compito, riuscendo in alcuni casi a produrre contenuti valutati positivamente in modo automatico.

4.6 Confronto RL vs modello base

Dopo l'addestramento, è stato effettuato un confronto tra modello base e modello RL su un sottoinsieme di prompt.

Prompt	**Reward Base**	**Reward RL**	**Wordplay**	**Offensive Humor**
"Joke about dieting"	0.12	1.74	6.9	0.0
"Joke about AI"	0.42	1.69	7.2	0.0
"9/11 joke"	0.70	2.14	0.0	9.2

Tabella 4.1: Confronto reward tra modello base e modello RL.

Come si nota, il modello RL è in grado di generare battute più mirate, spesso ottenendo punteggi molto alti in almeno una categoria, come richiesto dal reward design.
L'applicazione del Reinforcement Learning ha portato a miglioramenti qualitativi nella generazione delle battute, con un reward medio più elevato rispetto

al modello iniziale. Sebbene il processo sia soggetto a rumore e richieda fine-tuning della funzione di reward, la metodologia ha dimostrato efficacia nel generare contenuti più vicini al tipo di umorismo desiderato.
Il modello è stato infine salvato e compresso in un archivio ZIP per essere facilmente riutilizzato in fase di inferenza.

Capitolo 5

Conclusioni e sviluppi fututri

L'umorismo rappresenta una delle forme più complesse della comunicazione umana: è soggettivo, culturale, talvolta ambiguo, e difficilmente formalizzabile in metriche oggettive. Questo progetto ha affrontato il problema della generazione umoristica da parte degli LLM, evidenziandone i limiti e proponendo un'architettura modulare in grado di valutarne, categorizzarne e infine migliorarne la produzione.

Nel corso del lavoro abbiamo progettato e implementato una pipeline in tre fasi:

- una prima fase di categorizzazione automatica delle battute attraverso una *mixture* di LLM (Mistral, LLaMA3 e Gemma);
- una seconda fase di regressione supervisionata tramite il modello ALBERT, per ottenere una valutazione automatica e leggera delle categorie comiche;
- una terza fase di ottimizzazione generativa tramite Reinforcement Learning, dove il modello Mistral è stato adattato per produrre battute con punteggi elevati in almeno una delle categorie definite.

Un aspetto centrale del nostro contributo è stato l'introduzione della metrica LPR (Liparoti–Le Pera Ranking), un meccanismo di aggregazione del consenso tra modelli che ha permesso di superare la mancanza di annotazioni umane af-

fidabili. Questa metrica ha guidato l'addestramento sia del modello regressore, sia della funzione di ricompensa durante il fine-tuning con PPO.
Nonostante i limiti strutturali degli LLM, il nostro sistema ha mostrato la possibilità concreta di influenzare la produzione umoristica in maniera controllata. I risultati quantitativi (MSE, MAE, grafici) e qualitativi (confronto tra ground truth e predizioni) hanno evidenziato sia i punti di forza del sistema — come l'identificazione di contenuti potenzialmente offensivi o la coerenza con le categorie — sia le criticità, in particolare nella categoria "Wordplay", dove la creatività linguistica resta difficile da modellare.
Il fine-tuning tramite RL ha permesso di ottenere un miglioramento graduale nella qualità delle battute generate, secondo i criteri definiti dal nostro sistema di valutazione. Sebbene i risultati siano ancora perfettibili, il trend dei reward e il confronto con il modello base suggeriscono che l'approccio sia promettente.

Inoltre, riteniamo che sia degno di approfondimenti futuri il lavoro svolto per l'ottenimento di dati annotati in maniera automatica attraverso tecniche Self-supervised; l'approccio proposto, che è basato sull'utilizzo di Mixture di modelli differenti e metriche di consenso, potrebbe essere una valida alternativa all'annotazione manuale per diversi tipi di task.

In conclusione, questo lavoro dimostra che, pur partendo da un compito fortemente soggettivo e privo di benchmark solidi, è possibile costruire una pipeline efficace e modulare per la generazione controllata dell'umorismo. Le tecniche impiegate — dall'uso combinato di più LLM, all'introduzione di regressori leggeri, fino all'ottimizzazione tramite Reinforcement Learning — offrono spunti di generalizzazione verso altri task open-ended.

Nel capitolo successivo viene riportata una demo del sistema, che illustra in modo pratico l'utilizzo del modello risultante dal processo, come generatore di testo umoristico: un chatbot capace di interagire con l'utente producendo battute, risposte ironiche e commenti divertenti, mettendo in evidenza le potenzialità e i limiti dell'approccio proposto.

Capitolo 6

Demo del sistema

In questa sezione viene presentata una demo interattiva del modello sviluppato e ottimizzato tramite Reinforcement Learning con algoritmo PPO. L'obiettivo della demo è consentire all'utente di esplorare in modo pratico il funzionamento del generatore umoristico, mettendo in evidenza le capacità del modello di produrre battute e risposte ironiche in risposta a prompt testuali liberi.
La demo è stata realizzata come *notebook* Jupyter ed è progettata a scopo puramente dimostrativo per essere eseguita su Kaggle, se si dispone del modello finetunato in input, rendendone semplice l'esecuzione e la sperimentazione senza necessità di configurazioni locali particolari. All'avvio, il notebook carica il modello Mistral finetunato con i pesi LoRA ottenuti durante la fase di ottimizzazione con reward personalizzato.

6.1 Modalità di utilizzo

La demo supporta due modalità operative principali:

- **Generazione one-shot**: ogni richiesta dell'utente è trattata come indipendente; il modello non mantiene memoria dei messaggi precedenti e genera risposte basate esclusivamente sul prompt corrente.

- **Modalità conversazionale**: in questa modalità sperimentale viene mantenuto un breve contesto delle interazioni precedenti, consentendo al modello di produrre risposte più coerenti all'interno di una conversazione multi-turno.

La modalità predefinita della demo utilizza la generazione one-shot poichè il prompt su cui è stato fine-tunato il modello non prevede l'utilizzo del contesto precedente, e pertanto è questo l'unico caso d'uso in cui si riscontra un effettivo miglioramento del modello. Per i futuri sviluppi è possibile migliorare il modello attraverso fine-tuning mantenendo le risposte precedenti.

6.2 Esempi di interazione

Nel seguito si riportano alcuni esempi di prompt e risposte ottenute dal modello durante la demo.

Tu: Tell me a joke
Modello: I'd tell you a joke about time travel, but it's still in beta. It hasn't been debugged yet. But here's one about puddles: Why don't scientists trust atoms? Because they make up everything!

Figura 6.1: Esempio di interazione 1: risposta divertente a una richiesta standard.

Tu: What is the Colosseum?
Modello: Great question! It's where Romans went for their weekly dose of Wi-Fi.

Figura 6.2: Esempio di interazione 2: risposta ironica del modello a una richiesta fuori contesto.

Tu: Tell me a joke about italians
Modello: I once heard an Italian chef claim pasta is his ex-wife. Too many spaghetti at first, then a lasagna, and now he's just trying to fettuccine.

Figura 6.3: Esempio di interazione 3: battuta basata su stereotipi culturali.

Tu: Tell me a joke about Mattia, our disabled friend.

Modello: I once knew a man named Mattia, his jokes were so bad, even his echo couldn't give them a hand clap.

Figura 6.4: Esempio di interazione: battuta su un amico disabile.

Le interazioni evidenziano la capacità del modello di generare battute concise, ironiche e talvolta surreali, seguendo lo stile definito nella fase di addestramento. Si nota inoltre come il modello risponda in modo ironico e tagliente anche a richieste che esulano dall'ambito umoristico.

6.3 Accesso alla demo

Il notebook contenente la demo è disponibile al seguente link:

`https://www.kaggle.com/code/jacopolepera/nlp-demo`